Object Oriented Programming | **OOP Worksheet**
BS DS/CY — Fall Semester | *Practice Edition*

Name: _____ Roll No: _____ Section: _____

Time Allowed: 3 Hours **Total Marks: 100**

Instructions: Section A contains tricky dry-run questions. Write the exact output in the box on the right. If there is a compile error, write 'COMPILE ERROR' and state the reason. Section B contains problem-solving questions — read carefully and answer completely.

SECTION A: DRY RUN — Write the Exact Output [30 x 2 = 60 Marks]

Analyze each program. Write the exact output in the right-hand box. These are intentionally tricky — watch for vtables, slicing, default args, and constructor order.

Q1. [2 Marks] — Virtual + Default Arguments

```
#include<iostream>
using namespace std;
class B {
public:
  virtual void f(int x=1){
    cout<<"B"<<x<<endl;
  }
};
class D : public B {
public:
  void f(int x=2){
    cout<<"D"<<x<<endl;
  }
};
int main(){
  B* p = new D();
  p->f();
  D* d = new D();
  d->f();
  delete p; delete d;
}
```

Output:

Q2. [2 Marks] — Constructor Order — Multiple Inheritance

```
#include<iostream>
using namespace std;
struct X{X(){cout<<"X";}~X(){cout<<"~X";}};
struct Y{Y(){cout<<"Y";}~Y(){cout<<"~Y";}};
struct Z : Y, X {
  Z(){cout<<"Z";}
  ~Z(){cout<<"~Z";}
};
int main(){
  Z z;
  cout<<"|" ;
}
```

Output:

Q3. [2 Marks] — Virtual Called in Constructor

```
#include<iostream>
using namespace std;
class A {
public:
  A(){ show(); }
  virtual void show(){cout<<"A"<<endl;}
};
class B : public A {
public:
  B(){ show(); }
  void show(){cout<<"B"<<endl;}
};
class C : public B {
public:
  C(){ show(); }
  void show(){cout<<"C"<<endl;}
};
int main(){ C c; }
```

Output:

Q4. [2 Marks] — Static Member + Inheritance

```
#include<iostream>
using namespace std;
class Base{
public:
  static int n;
  Base(){ n++; }
  ~Base(){ n--; }
};
int Base::n=0;
class Child: public Base{};
int main(){
  Base a,b;
  Child c;
  cout<<Base::n<<endl;
  cout<<Child::n<<endl;
}
```

Output:

Q5. [2 Marks] — Object Slicing vs Pointer

```
#include<iostream>
using namespace std;
class A{
public:
 virtual void f(){cout<<"A"<<endl;}
};
class B:public A{
public:
 void f(){cout<<"B"<<endl;}
};
void byVal(A a){ a.f(); }
void byRef(A& a){ a.f(); }
void byPtr(A* a){ a->f(); }
int main(){
 B b;
 byVal(b);
 byRef(b);
 byPtr(&b);
}
```

Output:

Q6. [2 Marks] — Operator Overloading — Chaining

```
#include<iostream>
using namespace std;
class N{
  int v;
public:
  N(int v):v(v){}
  N operator+(N o){return N(v+o.v);}
  N operator*(N o){return N(v*o.v);}
  void p(){cout<<v<<endl;}
};
int main(){
  N a(2),b(3),c(4);
  (a+b*c).p();
  (a*b+c).p();
}
```

Output:

Q7. [2 Marks] — Non-Virtual Destructor Memory Leak

```
#include<iostream>
using namespace std;
class B{
public:
  B(){cout<<"B()"<<endl;}
  ~B(){cout<<"~B"<<endl;}
};
class D:public B{
public:
  D(){cout<<"D()"<<endl;}
  ~D(){cout<<"~D"<<endl;}
};
int main(){
  B* p = new D();
  delete p;
}
```

Output:

Q8. [2 Marks] — Diamond + Virtual Inheritance

```
#include<iostream>
using namespace std;
struct A{
  A(){cout<<"A";}
};
struct B: virtual A{
  B(){cout<<"B";}
};
struct C: virtual A{
  C(){cout<<"C";}
};
struct D: B, C{
  D(){cout<<"D";}
};
int main(){ D d; }
```

Output:

Q9. [2 Marks] — Prefix vs Postfix Tricky

```
#include<iostream>
using namespace std;
class C{
  int v;
public:
  C(int v):v(v){}
  C& operator++(){ v+=10; return *this; }
  C operator++(int){ C t=*this; v+=5; return t;
}
  void p(){cout<<v<<endl;}
};
int main(){
  C a(0);
  (a++).p();
  (++a).p();
  a.p();
}
```

Output:

Q10. [2 Marks] — Name Hiding vs Overriding

```
#include<iostream>
using namespace std;
class P{
public:
 void show(int x){cout<<"P"<<x<<endl;}
};
class Q: public P{
public:
 void show(){cout<<"Q"<<endl;}
};
int main(){
 Q q;
 q.show();
 // q.show(5); -- does this compile?
 P* p=&q;
 p->show(99);
}
```

Output:

Q11. [2 Marks] — Conversion Operator Ambiguity

```
#include<iostream>
using namespace std;
class Deg{
  double d;
public:
  Deg(double d):d(d){}
  operator double(){return d*3.14159/180;}
  void p(){cout<<d<<endl;}
};
int main(){
  Deg ang(180);
  ang.p();
  double r = ang;
  cout<<(int)r<<endl;
}
```

Output:

Q12. [2 Marks] — dynamic_cast returns nullptr

```
#include<iostream>
using namespace std;
class A{public: virtual ~A(){}};
class B: public A{};
class C: public A{};
int main(){
  A* a = new B();
  C* c = dynamic_cast<C*>(a);
  B* b = dynamic_cast<B*>(a);
  cout<<(c==nullptr?"null":"C")<<endl;
  cout<<(b==nullptr?"null":"B")<<endl;
  delete a;
}
```

Output:

Q13. [2 Marks] — Assignment Chaining

```
#include<iostream>
using namespace std;
class V{
  int x;
public:
  V(int x):x(x){}
  V& operator=(const V& o){
    x=o.x*2;
    cout<<x<<endl;
    return *this;
  }
};
int main(){
  V a(1),b(2),c(3);
  a=b=c;
}
```

Output:

Q14. [2 Marks] — Virtual Override Chain — no re-override

```
#include<iostream>
using namespace std;
class A{public: virtual void f(){cout<<"A";}};
class B:public A{public: void f(){cout<<"B";}};
class C:public B{}; // no override
class D:public C{public: void f(){cout<<"D";}};
int main(){
  A* p;
  p=new C(); p->f(); delete p;
  cout<<endl;
  p=new D(); p->f(); delete p;
  cout<<endl;
}
```

Output:

Q15. [2 Marks] — Const Member Function Overloading

```
#include<iostream>
using namespace std;
class Box{
  int v;
public:
  Box(int v):v(v){}
  void show(){cout<<"non-const:"<<v<<endl;}
  void show() const{cout<<"const:"<<v<<endl;}
};
void test(const Box& b){ b.show(); }
int main(){
  Box b(7);
  b.show();
  test(b);
}
```

Output:

Q16. [2 Marks] — Pure Virtual Destructor

```
#include<iostream>
using namespace std;
class Abs{
public:
  virtual ~Abs()=0;
};
Abs::~Abs(){cout<<"~Abs"<<endl;}
class Con: public Abs{
public:
  ~Con(){cout<<"~Con"<<endl;}
};
int main(){
  Abs* a=new Con();
  delete a;
}
```

Output:

Q17. [2 Marks] — this Pointer Chaining

```
#include<iostream>
using namespace std;
class R{
  int v=0;
public:
  R& add(int x){v+=x;return *this;}
  R& mul(int x){v*=x;return *this;}
  R& sub(int x){v-=x;return *this;}
  void p(){cout<<v<<endl;}
};
int main(){
  R r;
  r.add(3).mul(4).sub(2).add(10).p();
}
```

Output:

Q18. [2 Marks] — Copy Constructor with Inheritance

```
#include<iostream>
using namespace std;
class B{
public:
  B(){cout<<"B()";}
  B(const B&){cout<<"B(cp)";}
  ~B(){cout<<"~B";}
};
class D:public B{
public:
  D():B(){cout<<"D()";}
  D(const D& d):B(d){cout<<"D(cp)";}
  ~D(){cout<<"~D";}
};
int main(){
  D x;
  D y=x;
}
```

Output:

Q19. [2 Marks] — Vtable Dispatch — 3-level

```
#include<iostream>
using namespace std;
class A{public:virtual void
f(){cout<<"A";}virtual void g(){cout<<"gA";}};
class B:public A{public:void f(){cout<<"B";}};
class C:public B{public:void g(){cout<<"gC";}};
int main(){
  A* p=new C();
  p->f();
  cout<<"|" ;
  p->g();
  delete p;
}
```

Output:

Q20. [2 Marks] — Friend Function + Overloaded +

```
#include<iostream>
using namespace std;
class M{
  int r,c;
public:
  M(int r,int c):r(r),c(c){}
  friend M operator+(const M& a,const M& b){
    return M(a.r+b.r, a.c+b.c);
  }
  void p(){cout<<r<<","<<c<<endl;}
};
int main(){
  M a(1,2),b(3,4);
  M c=a+b;
  c.p();
  (a+b+c).p();
}
```

Output:

Q21. [2 Marks] — Inheritance + Static Dispatch

```
#include<iostream>
using namespace std;
class Base{
public:
  void nv(){cout<<"B::nv"<<endl;}
  virtual void vf(){cout<<"B::vf"<<endl;}
};
class Der:public Base{
public:
  void nv(){cout<<"D::nv"<<endl;}
  void vf(){cout<<"D::vf"<<endl;}
};
void test(Base& x){x.nv(); x.vf();}
int main(){
  Der d; test(d);
}
```

Output:

Q22. [2 Marks] — Operator [] Overloading

```
#include<iostream>
using namespace std;
class Arr{
  int d[4]={10,20,30,40};
public:
  int& operator[](int i){return d[i];}
};
int main(){
  Arr a;
  cout<<a[1]<<endl;
  a[1]=99;
  cout<<a[1]+a[0]<<endl;
}
```

Output:

Q23. [2 Marks] — Covariant Return Type

```
#include<iostream>
using namespace std;
class A{
public:
  virtual A* make(){
    cout<<"A::make";return new A();
  }
};
class B:public A{
public:
  B* make(){
    cout<<"B::make";return new B();
  }
};
int main(){
  A* p=new B();
  A* q=p->make();
  delete p; delete q;
}
```

Output:

Q24. [2 Marks] — Protected Inheritance Access

```
#include<iostream>
using namespace std;
class A{
protected:
  int x=5;
public:
  void show(){cout<<x<<endl;}
};
class B: protected A{
public:
  void print(){show();}
};
class C: public B{
public:
  void display(){print();}
};
int main(){
  C c; c.display();
}
```

Output:

Q25. [2 Marks] — Postfix Operator Returns Old Value

```
#include<iostream>
using namespace std;
class T{
  int v;
public:
  T(int v):v(v){}
  T operator++(int){
    T old=*this;
    v+=3;
    return old;
  }
  void p(){cout<<v<<endl;}
};
int main(){
  T a(0);
  T b=a++;
  a.p(); b.p();
  T c=a++;
  c.p(); a.p();
}
```

Output:

Q26. [2 Marks] — typeid RTTI

```
#include<iostream>
#include<typeinfo>
using namespace std;
class A{public:virtual ~A(){}};
class B:public A{};
int main(){
  A* p=new B();
  A* q=new A();
  cout<<typeid(*p).name()<<endl;
  cout<<typeid(*q).name()<<endl;
  cout<<(typeid(*p)==typeid(*q))<<endl;
  delete p; delete q;
}
```

Output:

Q27. [2 Marks] — Multiple Inherit — Ambiguity Resolved

```
#include<iostream>
using namespace std;
class X{public:void greet(){cout<<"X"<<endl;}};
class Y{public:void greet(){cout<<"Y"<<endl;}};
class Z:public X,public Y{
public:
  void greet(){X::greet();Y::greet();}
};
int main(){
  Z z;
  z.greet();
  z.X::greet();
}
```

Output:

Q28. [2 Marks] — Template Specialization

```
#include<iostream>
using namespace std;
template<typename T>
void show(T x){cout<<"T:"<<x<<endl;}
template<>
void show(int x){cout<<"int:"<<x*2<<endl;}
template<>
void show(char x){cout<<"char:"<<x<<endl;}
int main(){
  show(5);
  show(3.14);
  show('Z');
}
```

Output:

Q29. [2 Marks] — Abstract Class Instantiation

```
#include<iostream>
using namespace std;
class Shape{
public:
  virtual double area()=0;
  void print(){cout<<"Area="<<area()<<endl;}
};
class Sq:public Shape{
  double s;
public:
  Sq(double s):s(s){}
  double area(){return s*s;}
};
int main(){
  Shape* sh=new Sq(5);
  sh->print();
  delete sh;
}
```

Output:

Q30. [2 Marks] — Destructor Order — Complex

```
#include<iostream>
using namespace std;
class A{public:~A(){cout<<"~A";}};
class B{public:~B(){cout<<"~B";}};
class C:public B{
  A a;
public:
  ~C(){cout<<"~C";}
};
int main(){
  C c;
  cout<<"|" ;
}
```

Output:

Object Oriented Programming | **OOP Worksheet**
BS DS/CY — Fall Semester | *Practice Edition*

Name: _____ Roll No: _____ Section: _____

Time Allowed: 3 Hours **Total Marks: 100**

SECTION B: PROBLEM SOLVING [See marks per question]

Read each question carefully. Some questions provide partial code — complete it. Others ask for a full program. Write complete, compilable C++ code.

Q1. Complete the Missing Function [10 Marks]

Carefully study the following classes:

```
class Ball {
public:
  virtual void showColor() = 0;
};
class RedBall : public Ball {
public:
  void showColor(){ cout<<"Red\n"; }
};
class BlueBall : public Ball {
public:
  void showColor(){ cout<<"Blue\n"; }
};
class Bucket {
  Ball* balls[10];
public:
  Bucket(){
    for(int i=0;i<10;i++)
      balls[i]=(rand()%2==0)? new BlueBall():new RedBall();
  }
  int discardBlue();  // <-- Write this function
};
int main(){
  Bucket bkt;
  cout<<bkt.discardBlue()<<endl; // prints count of blue balls removed
  cout<<bkt.discardBlue()<<endl; // prints 0 (already removed)
}
```

Your Task: Write the function int Bucket::discardBlue() that: iterates over all 10 Ball pointers, uses dynamic_cast<BlueBall*> to detect blue balls, sets those pointers to nullptr (deleting them), and returns the count of blue balls discarded.

Write your answer below:

Q2. Add Operator Overloading to an Existing Class [10 Marks]

Given the following partial class definition:

```
class Matrix {
  int data[2][2];
public:
  Matrix(int a,int b,int c,int d){
    data[0][0]=a; data[0][1]=b;
    data[1][0]=c; data[1][1]=d;
  }
  // --- Overload operator+ here ---
  // --- Overload operator== here ---
  // --- Overload operator<< here (friend) ---
};
int main(){
  Matrix A(1,2,3,4), B(5,6,7,8);
  Matrix C = A + B;
  cout << C;
  cout << (A==B) << endl;
}
```

Your Task: Write the three operator overloads (operator+, operator==, operator<<) for the Matrix class above. operator+ does element-wise addition and returns a new Matrix. operator== compares all 4 elements. operator<< prints rows on separate lines.

Write your answer below:

Q3. Design a Class Hierarchy [15 Marks]

Write a complete C++ program that does the following:

(3 pts) Define a base class Employee with: string name, int id, and a pure virtual function double calculateSalary().

(4 pts) Derive class Manager (adds float bonus). calculateSalary() = bonus * 10 + id.

(4 pts) Derive class Developer (adds int linesOfCode). calculateSalary() = linesOfCode * 2 + id.

(4 pts) In main(), create an Employee* array of size 4 (2 Managers, 2 Developers). Call calculateSalary() on each polymorphically and print. Clean up memory.

Write your answer below:

Q4. Find the Bug and Fix It [8 Marks]

The following code compiles but has a serious runtime problem. Identify the issue, explain it, and rewrite the corrected version:

```
#include<iostream>
using namespace std;
class Resource {
  int* data;
public:
  Resource(){ data = new int[100]; cout<<"alloc"<<endl; }
  ~Resource(){ delete[] data; cout<<"free"<<endl; }
};
class Handle : public Resource {
  int id;
public:
  Handle(int i): id(i){ cout<<"Handle"<<endl; }
  ~Handle(){ cout<<"~Handle"<<endl; }
};
int main(){
  Resource* r = new Handle(42);
  delete r;
}
```

Your Task: a) State the exact bug (1-2 sentences). b) State the output you see vs the output you should see. c) Write the corrected code.

Write your answer below:

Q5. Predict Output Then Extend the Code [12 Marks]

Part A (4 pts): Write the exact output of the code below.

Part B (8 pts): Extend the code by adding a class C that inherits from both A and B using virtual inheritance to solve the diamond problem. Add a method introduce() in C that calls both A::speak() and B::speak(), then demonstrate it in main().

```
#include<iostream>
using namespace std;
class Base{
public:
  Base(){cout<<"Base()"<<endl;}
  virtual void speak(){cout<<"Base::speak"<<endl;}
  virtual ~Base(){cout<<"~Base"<<endl;}
};
class A : public Base{
public:
  A(){cout<<"A()"<<endl;}
  void speak(){cout<<"A::speak"<<endl;}
  ~A(){cout<<"~A"<<endl;}
};
class B : public Base{
public:
  B(){cout<<"B()"<<endl;}
  void speak(){cout<<"B::speak"<<endl;}
  ~B(){cout<<"~B"<<endl;}
};
int main(){
  Base* p = new A();
  p->speak();
  delete p;
}
```

Write your answer below:

Q6. Write the Missing Methods for a Generic Container [10 Marks]

Study the class below. Write the two missing methods as described.

```
class Stack {
  int* arr;
  int top;
  int cap;
public:
  Stack(int cap): cap(cap), top(-1){
    arr = new int[cap];
  }
  ~Stack(){ delete[] arr; }
  void push(int val);   // <-- write this
  int  pop();           // <-- write this
  bool isEmpty(){ return top==-1; }
  bool isFull() { return top==cap-1; }
  void print();         // <-- write this
};
int main(){
  Stack s(5);
  s.push(10); s.push(20); s.push(30);
  s.print();      // 30 20 10
  cout<<s.pop()<<endl;  // 30
  s.print();      // 20 10
}
```

Your Task: Write push(int val): if not full, increment top and store val. Write pop(): if not empty, return arr[top--], else return -1. Write print(): print from top to 0, space-separated.

Write your answer below:

Q7. Design a Polymorphic Notification System [15 Marks]

Write a complete C++ program that:

(3 pts) Abstract class Message with: string sender, pure virtual void send(), pure virtual string format().

(4 pts) Derive Email (adds: string to, string subject). send() prints formatted email. format() returns 'EMAIL: [subject] from [sender] to [to]'.

(4 pts) Derive SMS (adds: string phoneNumber). send() prints formatted SMS. format() returns 'SMS: from [sender] to [phoneNumber]'.

(4 pts) Class Inbox holds Message* array (size 10, track count). Methods: addMessage(Message*), sendAll() — calls send() on each. main() creates 2 Emails and 2 SMSes, adds to Inbox, calls sendAll(). Proper destructor.

Write your answer below:

Q8. Use RTTI to Write a Type-Aware Function [10 Marks]

Given the hierarchy below, write the function processAll():

```
class Animal{
public:
  string name;
  Animal(string n):name(n){}
  virtual ~Animal(){}
  virtual void speak()=0;
};
class Dog: public Animal{
public:
  Dog(string n):Animal(n){}
  void speak(){cout<<name<<": Woof"<<endl;}
  void fetch(){cout<<name<<" fetches!"<<endl;}
};
class Cat: public Animal{
public:
  Cat(string n):Animal(n){}
  void speak(){cout<<name<<": Meow"<<endl;}
  void purr(){cout<<name<<" purrs."<<endl;}
};
// Write: void processAll(Animal** arr, int n)
// For each animal: call speak(). If Dog, also call fetch().
// If Cat, also call purr().
```

Your Task: Write void processAll(Animal** arr, int n) using dynamic_cast to detect the actual type and call the appropriate type-specific method in addition to speak(). Then write a main() that creates an array of 4 animals (2 Dogs, 2 Cats) and calls processAll().

Write your answer below:

Object Oriented Programming | **OOP Worksheet**
BS DS/CY — Fall Semester | *Practice Edition — Set 2*

Name: _____ Roll No: _____ Section: _____

Time Allowed: 3 Hours **Total Marks: 100**

Instructions: Section A contains 35 tricky dry-run questions (2 marks each = 70 marks). Write the exact output in the right box. If there is a compile error, write 'COMPILE ERROR' with the reason. Section B contains 15 problem-solving questions — read carefully and answer completely.

SECTION A: DRY RUN — Write the Exact Output [35 x 2 = 70 Marks]

Analyze each program carefully. Write the exact output in the right-hand box. Watch for initializer order, slicing, lambdas, templates, and exception flows.

Q1. [2 Marks] — Initializer List Order vs Declaration Order

```
#include<iostream>
using namespace std;
class T{
  int a,b,c;
public:
  T():c(1),b(c+1),a(b+c){
    cout<<a<<" "<<b<<" "<<c<<endl;
  }
};
int main(){ T t; }
```

Output:

Q2. [2 Marks] — Static Local in Member Function

```
#include<iostream>
using namespace std;
class C{
public:
  void f(){
    static int x=0;
    x++;
    cout<<x<<endl;
  }
};
int main(){
  C a,b;
  a.f(); b.f(); a.f();
}
```

Output:

Q3. [2 Marks] — Overloaded << with Inheritance

```
#include<iostream>
using namespace std;
class A{
public:
  friend ostream& operator<<(ostream& os,const
A& a){
    return os<<"A";
  }
};
```

Output:

```
class B:public A{};
int main(){
  A a; B b;
  cout<<a<<" "<<b<<endl;
}
```

Q4. [2 Marks] — Exception in Constructor

```
#include<iostream>
using namespace std;
class R{
public:
  R(){cout<<"R()";}
  ~R(){cout<<"~R";}
};
class T{
  R r;
public:
  T(){cout<<"T()"<<endl; throw 1;}
  ~T(){cout<<"~T";}
};
int main(){
  try{ T t; }
  catch(...){ cout<<"caught"<<endl; }
}
```

Output:

Q5. [2 Marks] — Mutable Member in Const Function

```
#include<iostream>
using namespace std;
class C{
  mutable int count=0;
  int val;
public:
  C(int v):val(v){}
  int get() const{
    count++;
    return val;
  }
  void showCount(){cout<<count<<endl;}
};
int main(){
  const C c(5);
  cout<<c.get()<<endl;
  cout<<c.get()<<endl;
}
```

Output:

Q6. [2 Marks] — Virtual Base — Constructor Called Once

```
#include<iostream>
using namespace std;
struct V{
  int x;
  V(int x):x(x){cout<<"V"<<x;}
};
struct A:virtual V{ A():V(1){cout<<"A";} };
struct B:virtual V{ B():V(2){cout<<"B";} };
struct D:A,B{ D():V(99),A(),B(){cout<<"D";} };
int main(){ D d; cout<<" "<<d.x; }
```

Output:

Q7. [2 Marks] — Implicit Conversion Chain

```
#include<iostream>
using namespace std;
class A{
public:
  A(int x){cout<<"A("<<x<<")";}
};
void f(A a){cout<<"f"; }
int main(){
  f(42);
  cout<<endl;
}
```

Output:

Q8. [2 Marks] — Deleted Copy Constructor

```
#include<iostream>
using namespace std;
class NC{
public:
  NC(){}
  NC(const NC&)=delete;
  NC& operator=(const NC&)=delete;
};
int main(){
  NC a;
  NC b=a;  // line X
  cout<<"ok"<<endl;
}
```

Output:

Q9. [2 Marks] — Override + Final Keyword

```
#include<iostream>
using namespace std;
class A{public:virtual void f(){cout<<"A";}};
class B:public A{public:void f()
final{cout<<"B";}};
class C:public B{public:void f(){cout<<"C";}};
// line Y
int main(){
  B* p=new C();
  p->f();
}
```

Output:

Q10. [2 Marks] — Destructor Called on Array Delete

```
#include<iostream>
using namespace std;
class X{
  int id;
public:
  X(int i):id(i){cout<<"X"<<id;}
  ~X(){cout<<"~X"<<id;}
};
int main(){
  X* arr=new X[3]{X(1),X(2),X(3)};
  delete[] arr;
}
```

Output:

Q11. [2 Marks] — Nested Class Access

```
#include<iostream>
using namespace std;
class Outer{
  int x=10;
public:
  class Inner{
  public:
    void show(Outer& o){cout<<o.x<<endl;}
  };
};
int main(){
  Outer o;
  Outer::Inner i;
  i.show(o);
}
```

Output:

Q12. [2 Marks] — Rvalue Reference Overload

```
#include<iostream>
using namespace std;
void f(int& x) {cout<<"lval";}
void f(int&& x){cout<<"rval";}
int main(){
  int a=5;
  f(a);
  f(10);
  f(std::move(a));
}
```

Output:

Q13. [2 Marks] — Pure Virtual with Body

```
#include<iostream>
using namespace std;
class Abs{
public:
  virtual void f()=0;
};
void Abs::f(){cout<<"Abs::f";}
class Con:public Abs{
public:
  void f(){Abs::f(); cout<<" Con"<<endl;}
};
int main(){
  Con c; c.f();
}
```

Output:

Q14. [2 Marks] — Template Function Deduction Trap

```
#include<iostream>
using namespace std;
template<class T>
T bigger(T a, T b){ return a>b?a:b; }
int main(){
  cout<<bigger(3,4)<<endl;
  cout<<bigger(3.5,4.2)<<endl;
  cout<<bigger<int>(3.9,4.1)<<endl;
}
```

Output:

Q15. [2 Marks] — String Stream + Overloaded >>

```
#include<iostream>
#include<sstream>
using namespace std;
int main(){
  string s="10 20 30";
  istringstream ss(s);
  int a,b,c;
  ss>>a>>b>>c;
  cout<<a+b+c<<endl;
  cout<<(a<b && b<c)<<endl;
}
```

Output:

Q16. [2 Marks] — Placement new + Manual Destructor

```
#include<iostream>
using namespace std;
class T{
public:
  T(){cout<<"T()";}
  ~T(){cout<<"~T";}
};
int main(){
  char buf[sizeof(T)];
  T* p=new(buf) T();
  p->~T();
  cout<<"done";
}
```

Output:

Q17. [2 Marks] — Multiple Return Types via Conversion

```
#include<iostream>
using namespace std;
class Wrap{
  int v;
public:
  Wrap(int v):v(v){}
  operator int()  const{return v;}
  operator bool() const{return v!=0;}
  operator double()const{return v/2.0;}
};
int main(){
  Wrap w(7);
  cout<<(int)w<<endl;
  cout<<(bool)w<<endl;
  cout<<(double)w<<endl;
}
```

Output:

Q18. [2 Marks] — Partial Template Specialization

```
#include<iostream>
using namespace std;
template<class T,class U>
struct Pair{ void f(){cout<<"gen";} };
template<class T>
struct Pair<T,T>{ void f(){cout<<"same";} };
template<>
struct Pair<int,double>{ void f(){cout<<"id";}
};
int main(){
  Pair<int,char>  a; a.f();
  Pair<int,int>   b; b.f();
  Pair<int,double>c; c.f();
}
```

Output:

Q19. [2 Marks] — Chained Ternary with Side Effects

```
#include<iostream>
using namespace std;
int inc(int& x){return ++x;}
int dec(int& x){return --x;}
int main(){
  int a=5;
  int r = (a>3) ? inc(a) : dec(a);
  cout<<r<<" "<<a<<endl;
  r = (a<3) ? inc(a) : dec(a);
  cout<<r<<" "<<a<<endl;
}
```

Output:

Q20. [2 Marks] — Operator new Overload

```
#include<iostream>
using namespace std;
class M{
public:
  static void* operator new(size_t sz){
    cout<<"new("<<sz<<")";
    return ::operator new(sz);
  }
  static void operator delete(void* p){
    cout<<"del";
    ::operator delete(p);
  }
  M(){cout<<"M";}
  ~M(){cout<<"~M";}
};
int main(){ M* p=new M(); delete p; }
```

Output:

Q21. [2 Marks] — Lambda Capture Trap

```
#include<iostream>
using namespace std;
int main(){
  int x=10;
  auto f=[x]()mutable{ x+=5; cout<<x<<endl; };
  auto g=[&x]()      { x+=5; cout<<x<<endl; };
  f(); cout<<x<<endl;
  g(); cout<<x<<endl;
}
```

Output:

Q22. [2 Marks] — Signed/Unsigned Comparison UB

```
#include<iostream>
using namespace std;
int main(){
  int si=-1;
  unsigned int ui=1;
  cout<<(si<ui)<<endl;
  cout<<(si<(int)ui)<<endl;
}
```

Output:

Q23. [2 Marks] — Friend Class Access

```
#include<iostream>
using namespace std;
class B;
class A{
  int secret=42;
  friend class B;
};
class B{
public:
  void reveal(A& a){
    cout<<a.secret<<endl;
  }
};
int main(){
  A a; B b;
  b.reveal(a);
}
```

Output:

Q24. [2 Marks] — Virtual + Protected Destructor

```
#include<iostream>
using namespace std;
class Base{
protected:
  virtual ~Base(){cout<<"~Base";}
};
class Der:public Base{
public:
  ~Der(){cout<<"~Der";}
};
int main(){
  Base* p=new Der();
  delete p;
}
```

Output:

Q25. [2 Marks] — Array of Objects — Constructor/Destructor

```
#include<iostream>
using namespace std;
int n=0;
class C{
public:
  C(){cout<<"C"<<++n;}
  ~C(){cout<<"~C"<<n--;}
};
int main(){
  C arr[3];
  cout<<"|"<<endl;
}
```

Output:

Q26. [2 Marks] — Pointer-to-Member Function

```
#include<iostream>
using namespace std;
class X{
public:
  void a(){cout<<"a";}
  void b(){cout<<"b";}
};
int main(){
  void (X::*fp)()=&X::a;
  X obj;
  (obj.*fp)();
  fp=&X::b;
  (obj.*fp)();
  cout<<endl;
}
```

Output:

Q27. [2 Marks] — Explicit Constructor Prevents Implicit Conversion

```
#include<iostream>
using namespace std;
class Safe{
public:
  explicit Safe(int x){cout<<"Safe("<<x<<")";}
};
void use(Safe s){cout<<" used";}
int main(){
  Safe s(5);
  use(s);
  cout<<endl;
  use(10); // Does this compile?
}
```

Output:

Q28. [2 Marks] — Copy Elision / NRVO

```
#include<iostream>
using namespace std;
class W{
public:
  W(){cout<<"W()";}
  W(const W&){cout<<"W(cp)";}
  ~W(){cout<<"~W";}
};
W make(){
  W w;
  return w;
}
int main(){
  W x=make();
  cout<<"|"<<endl;
}
```

Output:

Q29. [2 Marks] — Inherited Constructor (using Base::Base)

```
#include<iostream>
using namespace std;
class Base{
public:
  Base(int x){cout<<"B("<<x<<")";}
};
class Der:public Base{
public:
  using Base::Base;
  void show(){cout<<"Der";}
};
int main(){
  Der d(42);
  d.show();
}
```

Output:

Q30. [2 Marks] — Volatile Member — Optimizer Trap

```
#include<iostream>
using namespace std;
class Reg{
  volatile int flag;
public:
  Reg():flag(0){}
  void set(){flag=1;}
  void check(){
    if(flag) cout<<"set"<<endl;
    else cout<<"unset"<<endl;
  }
};
int main(){
  Reg r;
  r.check();
  r.set();
  r.check();
}
```

Output:

Q31. [2 Marks] — Throw From Destructor

```
#include<iostream>
using namespace std;
class Bad{
public:
  ~Bad() noexcept(false){
    throw runtime_error("boom");
  }
};
int main(){
  try{
    Bad b;
  }catch(exception& e){
    cout<<e.what()<<endl;
  }
}
```

Output:

Q32. [2 Marks] — std::initializer_list Constructor Priority

```
#include<iostream>
#include<initializer_list>
using namespace std;
class K{
public:
  K(int a,int b){cout<<"(int,int)";}
  K(initializer_list<int> il){
    cout<<"init_list("<<il.size()<<")";
  }
};
int main(){
  K a(1,2);
  K b{1,2};
  K c{1,2,3,4};
  cout<<endl;
}
```

Output:

Q33. [2 Marks] — Recursive Virtual Call

```
#include<iostream>
using namespace std;
class A{
public:
  virtual int f(int n){
    if(n<=0) return 0;
    return n+f(n-1);
  }
};
class B:public A{
public:
  int f(int n){
    if(n<=0) return 1;
    return n*A::f(n-1);
  }
};
int main(){
  A* p=new B();
  cout<<p->f(4)<<endl;
  delete p;
}
```

Output:

Q34. [2 Marks] — Scope Resolution in Template

```
#include<iostream>
using namespace std;
int x=100;
template<class T>
struct S{
  int x=42;
  void show(){
    cout<<x<<" "<<::x<<endl;
  }
};
int main(){
  S<int> s;
  s.show();
  s.x=99;
  s.show();
}
```

Output:

Q35. [2 Marks] — Move Constructor vs Copy

```
#include<iostream>
using namespace std;
class Res{
public:
  Res(){cout<<"ctor";}
  Res(const Res&){cout<<"copy";}
  Res(Res&&){cout<<"move";}
  ~Res(){cout<<"dtor";}
};
int main(){
  Res a;
  Res b=a;
  Res c=std::move(a);
  cout<<"|"<<endl;
}
```

Output:

Object Oriented Programming | **OOP Worksheet**
BS DS/CY — Fall Semester | *Practice Edition — Set 2*

Name: _____ Roll No: _____ Section: _____

Time Allowed: 3 Hours **Total Marks: 100**

SECTION B: PROBLEM SOLVING [See marks per question — Total 30 Marks]

Read each question carefully. Complete partial code, design full programs, or find and fix bugs. Write clean, compilable C++ code.

Q1. Complete the Template Stack [8 Marks]

The following is a partial generic stack class. Complete the missing methods:

```
template<class T>
class Stack{
  T arr[50]; int top=-1;
public:
  void push(T val);     // <-- write this
  T    pop();           // <-- write this (return default T() if empty)
  T    peek() const;    // <-- write this (return top without removing)
  bool isEmpty(){ return top==-1; }
};
int main(){
  Stack<int> si;
  si.push(10); si.push(20); si.push(30);
  cout<<si.peek()<<endl;   // 30
  cout<<si.pop()<<endl;    // 30
  cout<<si.peek()<<endl;   // 20
  Stack<string> ss;
  ss.push("hello"); ss.push("world");
  cout<<ss.pop()<<endl;    // world
}
```

Your Task: Write push, pop, and peek. push: if top<49, increment top and store val. pop: if not empty, return arr[top--], else return T(). peek: if not empty return arr[top], else return T().

Write your answer below:

Q2. Implement the Rule of Three [10 Marks]

The class below manages dynamic memory but is missing the Rule of Three. Add all three:

```
class DynArray{
  int* data;
  int  size;
public:
  DynArray(int n): size(n){ data=new int[n]; }
  ~DynArray(); // write this
  DynArray(const DynArray& other); // write this (deep copy)
  DynArray& operator=(const DynArray& other); // write this
  void fill(int v){ for(int i=0;i<size;i++) data[i]=v; }
  void print(){ for(int i=0;i<size;i++) cout<<data[i]<<" "; cout<<endl; }
};
int main(){
  DynArray a(4); a.fill(7);
  DynArray b=a;   // copy constructor
  b.fill(3);
  a.print();  // 7 7 7 7 (deep copy, unchanged)
  b.print();  // 3 3 3 3
  DynArray c(2); c=a;  // assignment operator
  c.print();   // 7 7 7 7
}
```

Your Task: Write the destructor (delete[] data), copy constructor (allocate new array, copy elements), and copy assignment operator (handle self-assignment, deallocate old, allocate new, copy).

Write your answer below:

Q3. Design a Bank Account Hierarchy [12 Marks]

Write a complete C++ program:

(3 pts) Abstract base class Account: string owner, double balance, pure virtual double calcInterest().

(3 pts) SavingsAccount (adds: double rate). calcInterest() = balance * rate.

(3 pts) LoanAccount (adds: double penaltyRate). calcInterest() = balance * penaltyRate * 1.5.

(3 pts) main(): Account* array of 4 (2 Savings, 2 Loans). Print owner, balance, and interest for each polymorphically. Clean up.

Write your answer below:

Q4. Find and Fix the Deep Copy Bug [8 Marks]

This code runs but produces wrong output due to a shallow copy issue. Find the bug, explain it, and fix it:

```
#include<iostream>
using namespace std;
class Buffer{
  char* data;
public:
  Buffer(const char* s){
    data=new char[50];
    strcpy(data,s);
  }
  ~Buffer(){ delete[] data; }
  void change(char c){ data[0]=c; }
  void print(){ cout<<data<<endl; }
};
int main(){
  Buffer a("hello");
  Buffer b=a;        // problem here
  b.change('X');
  a.print();   // should print: hello
  b.print();   // should print: Xello
}
```

Your Task: a) Explain exactly why this crashes or gives wrong output. b) Write the corrected Buffer class with a proper copy constructor.

Write your answer below:

Q5. Write the Missing Friend Operators [10 Marks]

Given this class, write the three missing friend operators:

```
class Vec2{
  float x,y;
public:
  Vec2(float x,float y):x(x),y(y){}
  friend Vec2 operator+(const Vec2& a,const Vec2& b); // element-wise add
  friend Vec2 operator*(const Vec2& v,float s);       // scalar multiply
  friend bool  operator==(const Vec2& a,const Vec2& b);
  friend ostream& operator<<(ostream& os,const Vec2& v);// prints (x,y)
};
int main(){
  Vec2 a(1,2),b(3,4);
  cout<<a+b<<endl;     // (4,6)
  cout<<a*2.5f<<endl;  // (2.5,5)
  cout<<(a==b)<<endl;  // 0
}
```

Your Task: Implement all four friend functions. operator+ adds component-wise. operator* multiplies both components by the scalar. operator== returns true if both x and y match. operator<< prints in (x,y) format.

Write your answer below:

Q6. Design a Polymorphic Shape Area Calculator [15 Marks]

Write a complete C++ program that:

(3 pts) Abstract class Shape: string color, pure virtual double area(), virtual void describe() prints color and area.

(4 pts) Circle (adds: double radius). area() = 3.14159 * r * r.

(4 pts) Rectangle (adds: double width, height). area() = width * height.

(4 pts) Triangle (adds: double base, height). area() = 0.5 * base * height.

Bonus: main() creates a Shape* array with 2 of each. Calls describe() on all. Finds and prints the shape with maximum area.

Write your answer below:

Q7. Implement Iterator Pattern with Operator Overloading [10 Marks]

Given the following class, write the missing operator overloads to make it work as an iterator:

```
class Range{
  int cur, end_;
public:
  Range(int start,int end):cur(start),end_(end){}
  Range begin(){ return *this; }
  Range end()  { return Range(end_,end_); }
  bool operator!=(const Range& o)const; // <-- write
  Range& operator++();                  // prefix <-- write
  int operator*() const;                // dereference <-- write
};
int main(){
  for(int x : Range(1,6))
    cout<<x<<" ";   // 1 2 3 4 5
  cout<<endl;
}
```

Your Task: Write operator!= (returns cur != o.cur), prefix operator++ (increments cur, returns *this), and operator* (returns cur). The range-for loop syntax requires all three.

Write your answer below:

Q8. Fix the Virtual Destructor and Memory Leak [8 Marks]

The program below has two distinct bugs. Identify both, explain, and write the corrected version:

```
#include<iostream>
using namespace std;
class Node{
public:
  int val;
  Node* next;
  Node(int v):val(v),next(nullptr){ cout<<"N("<<v<<")"; }
  ~Node(){ cout<<"~N("<<val<<")"; }
};
class List{
  Node* head=nullptr;
public:
  void add(int v){
    Node* n=new Node(v);
    n->next=head; head=n;
  }
  void print(){
    Node* c=head;
    while(c){ cout<<c->val<<" "; c=c->next; }
    cout<<endl;
  }
  // missing destructor
};
int main(){
  List L;
  L.add(1);L.add(2);L.add(3);
  L.print();
}
```

Your Task: a) What happens to the Node objects when List goes out of scope? b) Write the destructor for List that properly frees all Nodes. c) Write the complete corrected program.

Write your answer below:

Q9. Design a Publish-Subscribe Event System [12 Marks]

Write a complete C++ program:

(3 pts) Abstract class Listener with pure virtual void onEvent(string msg).

(4 pts) Derive Logger: onEvent prints '[LOG] ' + msg. Derive Alerter: onEvent prints '[ALERT] ' + msg.

(4 pts) Class EventBus holds Listener* array (max 10). Methods: subscribe(Listener*), publish(string) — calls onEvent on all subscribers.

(1 pt) main() creates 1 Logger and 2 Alerters, subscribes all, publishes 2 different messages. Show all 6 outputs.

Write your answer below:

Q10. Complete a Linked List with Templates [10 Marks]

The partial template linked list below needs its missing methods:

```
template<class T>
class LList{
  struct Node{ T val; Node* next; Node(T v):val(v),next(nullptr){} };
  Node* head=nullptr;
public:
  void pushFront(T v);    // <-- write
  T    popFront();        // <-- write (return T() if empty)
  bool contains(T v);     // <-- write
  ~LList();               // <-- write (delete all nodes)
};
int main(){
  LList<int> L;
  L.pushFront(3);L.pushFront(2);L.pushFront(1);
  cout<<L.contains(2)<<endl; // 1
  cout<<L.popFront()<<endl;  // 1
  cout<<L.contains(1)<<endl; // 0
}
```

Your Task: Write all four methods: pushFront creates a new Node, sets next=head, head=new node. popFront removes head and returns its val. contains traverses and returns true if found. Destructor deletes all nodes.

Write your answer below:

Q11. Write a Recursive Template Function [8 Marks]

Complete the following template functions:

```
// Returns the factorial of n (base case: n==0 returns 1)
template<class T>
T factorial(T n);  // <-- write

// Returns the power base^exp (base case: exp==0 returns 1)
template<class T>
T power(T base, int exp); // <-- write

// Returns sum of all elements in array
template<class T>
T sumArray(T* arr, int n); // <-- write (recursive, base: n==0 return 0)

int main(){
  cout<<factorial(5)<<endl;      // 120
  cout<<factorial(10LL)<<endl;   // 3628800
  cout<<power(2,10)<<endl;       // 1024
  cout<<power(3.0,4)<<endl;      // 81
  int a[]={1,2,3,4,5};
  cout<<sumArray(a,5)<<endl;     // 15
}
```

Your Task: Write all three template functions. Each should be recursive. factorial(0)=1, factorial(n)=n*factorial(n-1). power(b,0)=1, power(b,e)=b*power(b,e-1). sumArray with n==0 returns 0, else arr[0]+sumArray(arr+1,n-1).

Write your answer below:

Q12. Design a Smart Pointer Class [15 Marks]

Write a complete generic smart pointer class (similar to unique_ptr):

(3 pts) Template class SmartPtr<T>: holds T* ptr. Constructor takes raw T*, destructor deletes ptr.

(4 pts) Overload operator* (dereference, returns T&) and operator-> (returns T*). Delete copy constructor and copy assignment.

(4 pts) Add move constructor (SmartPtr(SmartPtr&&)) and move assignment. After move, source ptr becomes nullptr.

(4 pts) Demonstrate in main(): SmartPtr<int>, SmartPtr<string>, move semantics. Show that accessing moved-from pointer is safe (check for nullptr).

Write your answer below:

Q13. Predict and Extend — Exception Safety [10 Marks]

Part A (3 pts): Write the exact output.

Part B (7 pts): Extend the code — add a class SafeDiv that takes two ints, throws a runtime_error if divisor is zero, otherwise has a method result() that returns the division. Show usage with try/catch in main().

```
#include<iostream>
#include<stdexcept>
using namespace std;
class Calc{
  int a,b;
public:
  Calc(int a,int b):a(a),b(b){cout<<"Calc()"<<endl;}
  ~Calc(){cout<<"~Calc()"<<endl;}
  int add(){return a+b;}
  int div(){
    if(b==0) throw runtime_error("div/0");
    return a/b;
  }
};
int main(){
  try{
    Calc c1(10,2);
    cout<<c1.div()<<endl;
    Calc c2(5,0);
    cout<<c2.div()<<endl;
  }catch(exception& e){
    cout<<e.what()<<endl;
  }
}
```

Write your answer below:

Q14. Write a Generic Sorting Function with Comparator [8 Marks]

Complete the template bubble sort function:

```
template<class T, class Comp>
void bubbleSort(T* arr, int n, Comp cmp);  // <-- write this

int main(){
  int a[]={5,2,8,1,9,3};
  int n=6;
  // Sort ascending
  bubbleSort(a,n,[](int x,int y){return x<y;});
  for(int i=0;i<n;i++) cout<<a[i]<<" ";
  cout<<endl; // 1 2 3 5 8 9

  // Sort descending
  bubbleSort(a,n,[](int x,int y){return x>y;});
  for(int i=0;i<n;i++) cout<<a[i]<<" ";
  cout<<endl; // 9 8 5 3 2 1

  string s[]={"banana","apple","cherry"};
  bubbleSort(s,3,[](string a,string b){return a<b;});
  for(auto& x:s) cout<<x<<" ";
  cout<<endl; // apple banana cherry
}
```

Your Task: Write the bubbleSort template function. Use a nested loop: outer from 0 to n-1, inner from 0 to n-outer-2. If cmp(arr[j+1], arr[j]) is true, swap arr[j] and arr[j+1] using std::swap.

Write your answer below:

Q15. Design a Full Observer Pattern [12 Marks]

Write a complete C++ implementation of the Observer design pattern:

(3 pts) Abstract Observer class with pure virtual void update(int value).

(4 pts) Concrete Subject class: holds int state, Observer* list (max 10), int count. Methods: attach(Observer*), detach(Observer*), setState(int) — updates state and calls notify().

(4 pts) notify() calls update(state) on all observers. Derive ConcreteObserverA (prints 'A got: value') and ConcreteObserverB (prints 'B got: value*2').

(1 pt) main() creates Subject s, attaches 1A and 2B. Calls setState(5) then setState(10). Show all outputs.

Write your answer below: